# Face Recognition System

A complete beginner's guide for Windows
Installing, taking photos, training the model, and running live recognition

Prepared for first-time users

2026-09-29

1.0

# Table of Contents

**1. What This Project Does** ...

**2. What You Need Before You Start** ...

**3. Part 1 — Install Python** ...

**4. Part 2 — Put the Project Files on Your Laptop** ...

**5. Part 3 — Open the Command Window in the Right Place** ...

**6. Part 4 — Install the Library the Program Needs** ...

**7. Part 5 — Take the Photos (Step 1 of the program)** ...

**8. Part 6 — Train the Model (Step 2 of the program)** ...

**9. Part 7 — Recognise Faces Live (Step 3 of the program)** ...

**10. Using Your Own Photos Instead of the Webcam** ...

**11. Adding More People Later** ...

**12. Where Everything Is Saved** ...

**13. If Something Goes Wrong (Troubleshooting)** ...

**14. Quick Command Reference** ...

**15. How It Works (for your viva)** ...

**16. Final Checklist Before You Submit** ...

**17. Optional: Let an AI Assistant Set It All Up** ...

## 1. What This Project Does

This program recognises faces using your laptop's webcam. You teach it the faces of the people you want it to know, and after that it tells you who is standing in front of the camera — live, on screen.

The whole project is only three steps, and you do them in this order, one time:

1. Take photos of each person's face with the webcam.
2. Train (teach) the model using those photos.
3. Run the program and let it recognise faces live.

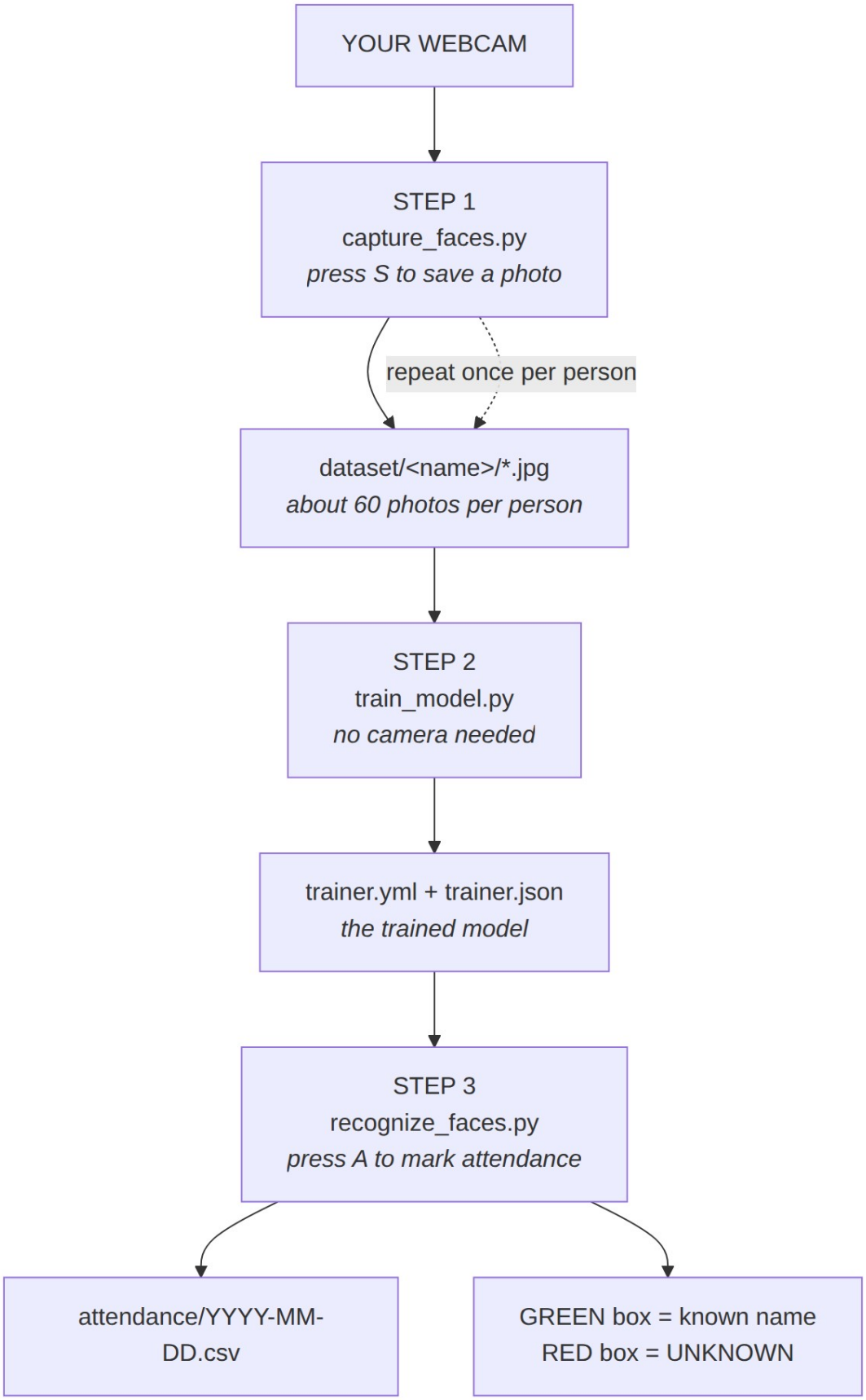



*Figure 1.1 — The three steps, and what each one produces.*

Important: the order matters. You cannot skip step 1, because step 2 has nothing to learn from. You only need to do steps 1 and 2 once for each new person. Step 3 is the one you run whenever you want to use the program.

Everything is controlled from a small black window called the Command Prompt. You type one short line, press Enter, and the program runs. This guide shows you exactly what to type.

If you would rather have an AI assistant do the typing for you, skip to Chapter 17 — that route needs no commands at all.

## 2. What You Need Before You Start

Check you have all five of these. Nothing here is expensive.

**Table 2.1 — Checklist before you begin.**

| What you need | Why you need it | Do you have it? |
|---|---|---|
| A Windows laptop (Windows 10 or 11) | The program runs on Windows. | |
| A working webcam | Both taking photos and recognising need it. | |
| An internet connection | Only for the one-time Python and library install (about 200 MB). | |
| About 20 minutes | 10 min setup, then photos and training. | |
| A quiet, well-lit room | Photos taken in bad light will not be recognised later. | |

How to check your webcam works: open the Windows Start menu, type Camera, and press Enter. If you see yourself, the webcam works. Close that window afterwards — you cannot run the program while the Camera app is using the webcam.

If Windows asks for permission to use the camera, choose Allow. If you clicked Block by mistake, go to Settings → Privacy → Camera and turn it on.

## 3. Part 1 — Install Python

Python is the language this project is written in. It is not installed on most Windows computers by default, so this is the first thing to do. You only do it once.

Step 1. Go to https://www.python.org/downloads/ and click the big yellow button that says Download Python 3.x for Windows.

Step 2. Open the file you just downloaded (it will be in your Downloads folder, named something like python-3.12.x-amd64.exe).

Step 3. THIS IS THE MOST IMPORTANT PART. On the first screen of the installer, at the very bottom, there is a ticked checkbox that says "Add python.exe to PATH". Leave it ticked. If you untick it, nothing in this guide will work and you will have to start over.

**Table 3.1 — The checkbox that decides whether this guide works.**

| Checkbox at the bottom of the installer | Ticked (correct) | Unticked (wrong) |
|---|---|---|
| Add python.exe to PATH | Click Install now | Nothing will work later. Go back and tick it. |

Step 4. Wait for it to finish. A black window flashes and closes. That is normal and means it worked.

Step 5. Check it worked. Press the Windows key, type cmd, press Enter. A black window opens. Type this and press Enter:

```
python --version
```

You should see something like Python 3.12.4. The exact numbers do not matter. If you see that, Python is installed correctly — close this window and continue to Part 2.

If instead you see a red message saying 'python' is not recognized, follow the fix in Chapter 13, Problem 1.

## 4. Part 2 — Put the Project Files on Your Laptop

You were given a file called face-recognition-for-friend.zip. It contains the program. You must unpack it before you can use it — Windows cannot run a program from inside a zip file.

Step 1. Put the zip somewhere easy to find, such as your Documents folder.

Step 2. Click the zip file once to select it. Then right-click it and choose "Extract All..." from the menu.

Step 3. In the window that appears, click the Extract button.

Step 4. Windows creates a folder called face-recognition. Open it. You should see exactly these six files:

**Table 4.1 — The six files you received.**

| File | What it does |
|---|---|
| capture_faces.py | Step 1. Takes the photos. |
| train_model.py | Step 2. Trains the model. |
| recognize_faces.py | Step 3. Recognises faces live. |
| config.py | Settings shared by all three. You normally do not edit it. |
| requirements.txt | The list of libraries to install. |
| README.md | A short reference. This guide is longer and covers more. |

IMPORTANT — where you put this folder. Move the whole face-recognition folder to the top of your C: drive, so the full path is exactly C:\face-recognition. Do not run it from the Desktop, from Documents, or from any folder whose name has a space in it (like "My Pictures"). Spaces in the path are the single most common cause of strange errors on Windows. To move it: open File Explorer, click This PC, click Local Disk (C:), then drag the face-recognition folder into this window.

## 5. Part 3 — Open the Command Window in the Right Place

From now on you type commands in a black window. The one rule that trips up beginners is this: the window must be opened inside the project folder, otherwise the computer will not know where to look for the files.

The easiest way, which always works:

4. Open File Explorer and go to C:\face-recognition.
5. Click once on the empty space to the right of the files, so no file is selected.
6. Now click the address bar at the very top (the bar showing the path). The whole path turns blue.
7. Type: cmd and press Enter.

A black window opens. Look at the text near the top-left. It must end with the name face-recognition, something like:

```
C:\face-recognition>
```

If it says C:\Users\YourName> instead, you opened it in the wrong place. Close the window and repeat the four steps above.

From now on, every command in this guide is typed into this window, followed by pressing the Enter key. Keep this window open for the whole session.

## 6. Part 4 — Install the Library the Program Needs

The program uses a free library called OpenCV that does the image and face work. You install it once, with a single command.

In the black window, type this and press Enter:

```
python -m pip install -r requirements.txt
```

This will take a few minutes and the screen will fill with text. That is normal — leave it alone until it finishes and returns you to the C:\face-recognition> prompt. Do not close the window.

We are using -m pip rather than just pip on purpose: it guarantees the library is installed into the same Python that you are running, which prevents a confusing class of errors.

What is requirements.txt? It is a list of libraries the program needs. The -r flag means "read that list and install everything in it". You never need to open or edit this file.

Two details in that line matter, and both are deliberate:

- It installs opencv-contrib-python, NOT opencv-python. The face recogniser this project uses lives only in the "contrib" version. If you install the ordinary one by mistake, the program crashes.
- It is limited to version 4 (written <5 in the file). Version 5 of that library was released without the face-detection data files the program needs, so version 5 will not work. The <5 in requirements.txt stops the wrong version from being installed automatically.

Check that it worked. Type:

```
python -c "import cv2; print(cv2.__version__)"
```

You should see a number starting with 4, for example 4.14.0. If you see a 5.x here, something went wrong — see Chapter 13, Problem 5.

## 7. Part 5 — Take the Photos (Step 1 of the program)

The program cannot recognise a face it has never seen. So first we take photos of each person and save them.

Step 1. Make sure the Windows Camera app is closed, otherwise the webcam is already busy and the program cannot open it.

Step 2. In your command window, type this and press Enter:

```
python capture_faces.py
```

Step 3. The program asks for a name. Type the person's name with no spaces, for example:

```
ali
```

This name becomes the label shown on screen later, and it is also the name of a new folder the program creates. Use the same name every time for the same person.

Step 4. A window opens showing your webcam. You will see a green box drawn around your face. If there is no green box, see Chapter 13, Problem 9.

**Table 7.1 — The only two keys you need.**

| Key | What it does |
|---|---|
| S | Saves one photo of your face. Press it again for the next photo. |
| Q | Quits and saves your work. Press this when you have enough photos. |

IMPORTANT: click on the camera window first so it has focus, then press the S key on your keyboard. If you press S while the black command window has focus, nothing will happen.

Step 5. Press S about 60 times, moving your head slightly between presses — a little left, a little right, slightly up and down. Variety is what makes recognition work. The counter in the top-left corner goes up each time.

Step 6. Press Q to finish this person.

Step 7. Repeat steps 2 to 6 for every other person you want the program to know. One full run per person.

For good results, while taking photos:

- Face the camera straight on, and keep your whole face inside the green box.
- Use a well-lit room. Light from a window in front of you is best; a bright light behind you makes a dark face.
- Remove sunglasses, hats and face coverings.
- Keep the background plain and uncluttered where you can.
- Do not blur or hide the photo files afterwards.

The program warns you if you quit with fewer than 40 photos, because the model will be weak. If you see that warning, just run the same command again with the same name — it adds more photos rather than replacing them.

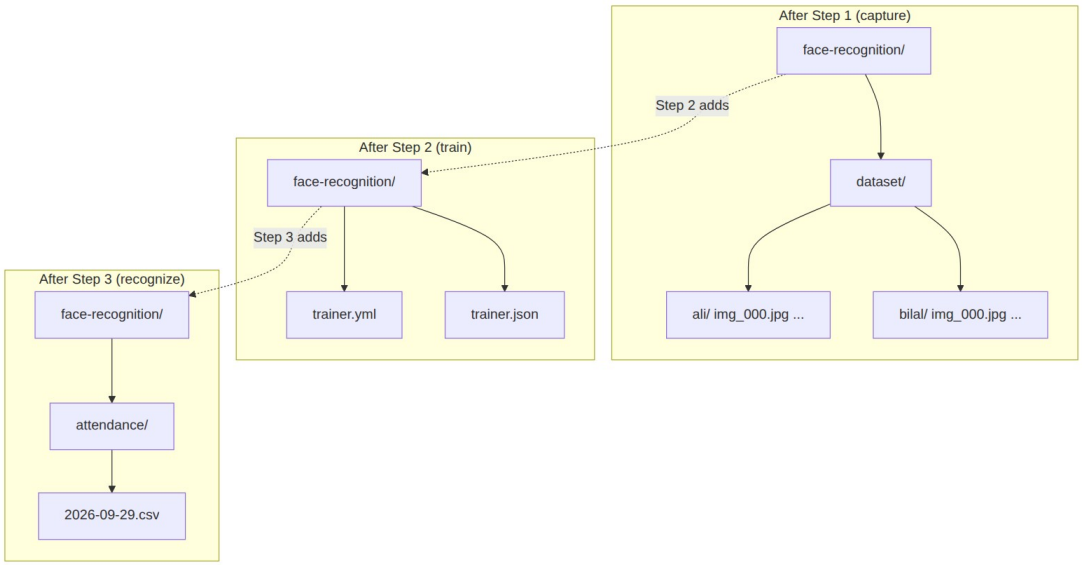



*Figure 7.2 — The dataset folders that Step 1 creates for you.*

## 8. Part 6 — Train the Model (Step 2 of the program)

Training is where the computer learns what each person's face looks like. This is the step your college project is really about.

This step does not use the webcam. Close the capture window if it is still open, then type:

```
python train_model.py
```

It usually finishes in a few seconds. You will see output like this:

```
Loading photos from C:\face-recognition\dataset
  ali: 60 photos
  bilal: 58 photos
Loaded 118 photos of 2 person(s).
Model saved to C:\face-recognition\trainer.yml
Labels saved to C:\face-recognition\trainer.json
Classes: {0: 'ali', 1: 'bilal'}
```

What those lines mean:

**Table 8.1 — Reading the training output.**

| Line | Meaning |
|---|---|
| ali: 60 photos | It found 60 usable photos for ali. Check these numbers — if a person shows 0, their folder is empty or in the wrong place. |
| Loaded 118 photos of 2 person(s) | 118 photos from 2 different people will be used for training. |
| Model saved to trainer.yml | The trained model. This is the important output. |
| Labels saved to trainer.json | The list of names, so the program knows that number 0 means 'ali'. |
| Classes: {0: 'ali', ...} | The name-to-number map. Each person gets one number. |

You can close the command window after this. Re-open it the same way as before (Chapter 5) for the next step.

When to train again: any time you add new photos or add a new person. The model is not updated automatically — if you take more photos and then go straight to Step 3, the program will still be using the old model.

## 9. Part 7 — Recognise Faces Live (Step 3 of the program)

This is the part you show to your class. Type:

```
python recognize_faces.py
```

A window opens showing the live camera with boxes drawn around every face it finds.

**Table 9.1 — What the colours mean.**

| What you see | What it means |
|---|---|
| GREEN box with a name and a percentage | The program recognised this person. For example: ali 68% |
| RED box saying UNKNOWN | This face is not in the training data, or the match was not close enough. See Chapter 13, |

| | |
|---|---|
| | Problem 11. |
| No box at all | No face was detected. Move closer and improve the lighting. |

**Table 9.2 — The keys in the recognition window.**

| Key | What it does |
|---|---|
| A | Marks the person currently on screen as present today, and writes a row to the attendance file. Press it once per person per day — pressing it twice for the same person will not create a duplicate. |
| Q | Quits the program. |

Click on the camera window first so it has focus, then press A.

Attendance is saved as a normal spreadsheet file you can open in Excel:

```
face-recognition\attendance\2026-09-29.csv
```

The file is named after today's date, so each day gets its own file. It has two columns: name and time.

To finish, press Q. The program closes cleanly and the command window shows "Stopped."

## 10. Using Your Own Photos Instead of the Webcam

If you would rather use pictures you already have, you can skip Chapter 7 entirely.

Step 1. Create a folder for each person inside the dataset folder, and name the folder after the person. The folder must be called exactly dataset, and the person folders must sit directly inside it.

**Table 10.1 — How to lay out your own photos.**

| Folder | Contents |
|---|---|
| face-recognition\dataset\ali\ | ali1.jpg, ali2.jpg, ali3.jpg, ... |
| face-recognition\dataset\bilal\ | bilal1.jpg, bilal2.jpg, ... |

Step 2. Copy your photos into the matching person's folder. They must be .jpg files (or .jpeg). If your photos are .png or .heic, convert or re-save them as .jpg first.

Step 3. Train, exactly as in Chapter 8:

```
python train_model.py
```

The training program automatically finds the face in each photo and crops to it. So ordinary pictures — ones with a background, or several people in them — work correctly. It always uses the largest face it finds in each photo.

Rules for choosing photos:

- One folder per person. Folder name = the name that will appear on screen. No spaces.
- 20 or more clear, front-facing, well-lit photos per person. More is better.
- A photo where the face is small, blurry, or half-covered teaches the model nothing useful. Delete bad photos.
- If no face can be found in a photo, the program uses the whole photo anyway, so a photo of a wall or a landscape will make recognition worse. Delete those.

## 11. Adding More People Later

You are not limited to one or two people. Any number works. Each folder inside dataset becomes one more person automatically.

To add someone, either run the capture program again with their name (Chapter 7), or drop their photos into a new folder (Chapter 10). Then run the training command again:

```
python train_model.py
```

You must retrain after every change. Forgetting this is the most common reason someone says "I added the photos but it still says UNKNOWN".

A realistic note on how many people: this method stays reliable with roughly 10 to 15 people, provided each has enough good photos. Beyond that, recognition accuracy starts to drop, and the fix is more photos per person rather than more people.

## 12. Where Everything Is Saved

**Table 12.1 — The four things the program creates.**

| Path | What it is | When it appears |
|---|---|---|
| dataset\<name>*.jpg | The photos you collected for one person | After Step 1 (or you create it) |
| trainer.yml | The trained model — the important file | After Step 2 |
| trainer.json | The number-to-name list | After Step 2 |
| attendance\<date>.csv | Who was marked present, and when | After Step 3, when you press A |

If you want to start completely fresh, delete the dataset, trainer.yml and trainer.json folders/files, then begin again from Chapter 7. Keep a copy of your photos somewhere safe first — deleting is permanent.

## 13. If Something Goes Wrong (Troubleshooting)

Find your error message in the left column. These are real messages the program produces.

**Table 13.1 — Every common error and its fix.**

| Problem (the message you see) | What it means | What to do |
|---|---|---|
| 'python' is not recognized... | Python is not installed, or was installed without the PATH box ticked. | Reinstall Python from python.org and TICK "Add python.exe to PATH" on the first screen. See Chapter 13 note below. |
| 'pip' is not recognized | The pip command is not on your path. | Do not use pip. Use this instead: python -m pip install -r requirements.txt |
| No module named 'cv2' | The library was not installed. | Run the install command in Chapter 6 again, and wait for it to fully finish. |
| module 'cv2' has no attribute 'face' | The wrong OpenCV was installed (opencv-python instead of opencv-contrib-python). | Run: python -m pip uninstall opencv-python opencv-contrib-python then run the Chapter 6 install command again. |
| No face cascade found | OpenCV version 5 was installed. Version 5 does not include the face files this program needs. | Run: python -m pip install "opencv-contrib-python>=4.5,<5" and check with the version command in Chapter 6. |
| Could not open the webcam. Check CAMERA_INDEX | The program could not take control of the camera. | Close the Windows Camera app and any video call. Then check Settings → Privacy → Camera is allowed. If you have both a built-in and an external camera, try changing CAMERA_INDEX = 0 to 1 in config.py. |
| No dataset folder. Run capture_faces.py first | You tried to train before taking any photos. | Do Chapter 7 first. |
| No face photos found in the dataset folder | The dataset folder exists but the photos inside it are missing or not .jpg files. | Check that your photos are directly inside dataset\<name>\ and end in .jpg. |
| No box appears around my face in the capture window | The camera is too far away, too dark, or your face is not facing it. | Move closer so your face is clearly bigger than a small box, face the camera, and turn on more light. |
| Everything shows red UNKNOWN | The face is not in the training data, or the photos are poor. | First make sure you ran train_model.py AFTER taking photos. Then take more, better-lit photos and retrain. If you are testing yourself and it says UNKNOWN, that means the model was not trained on your photos. |
| The command window says it cannot find the file | You opened the command window in the wrong folder. | Close it and redo Chapter 5, so the prompt shows C:\face-recognition> |
| Windows Security or antivirus | Antivirus sometimes | Add the project folder to your |

| | | |
|---|---|---|
| warns about the file | quarantines programs that use webcams. | antivirus's exclusion list, or temporarily disable it while you work. Only do this for this specific trusted folder. |

Note on Problem 1: if you already installed Python without the PATH box ticked, the simplest repair is to uninstall it (Settings → Apps → Python 3.x → Uninstall) and install it again with the box ticked. That is faster and less confusing than editing system paths by hand.

## 14. Quick Command Reference

Print this page. These are the only five commands you need. Run each one in the black command window opened in the project folder.

**Table 14.1 — The five commands.**

| When | Type this | What happens |
|---|---|---|
| One time only, Chapter 6 | python -m pip install -r requirements.txt | Installs the library. Takes a few minutes. |
| Step 1, per person | python capture_faces.py | Saves photos. Press S to save, Q to quit. |
| Step 2, after every change | python train_model.py | Trains the model. A few seconds. |
| Step 3, to use it | python recognize_faces.py | Runs live. Press A for attendance, Q to quit. |
| Check the version | python --version | Confirms Python is installed. |

## 15. How It Works (for your viva)

You may be asked to explain the project. Here is the short version, in your own words.

Stage 1 — Detection. OpenCV ships with a pre-trained file that knows what the general shape of a front-facing human face looks like. Every frame from the webcam is scanned with it, and a box is drawn around each region that looks like a face. Faces smaller than a set size are ignored, so the program does not try to recognise people standing far away.

Stage 2 — Training. The algorithm used is LBPH — Local Binary Patterns Histograms. It describes a face as a grid of small squares (8 by 8), and in each square it counts how the brightness of each pixel compares with its neighbours, producing a small histogram. The full face is therefore reduced to a short numerical signature. The program does this for every photo of every person, and stores one signature per person. The name attached to each signature is kept separately in a small text file, because the model itself only stores numbers.

Stage 3 — Recognition. For a new face, the same signature is calculated and compared with all the stored ones. The closest match wins. The result is reported as a confidence value, which is a distance, not a percentage — so LOWER IS BETTER. A distance below the

limit (70 in the settings) is accepted and the name is shown; a distance above the limit means the program is not confident, and it shows UNKNOWN instead.

The code is split into three separate files, one per stage, plus one config.py holding the shared settings. Splitting it this way means each stage can be run and checked on its own.

## 16. Final Checklist Before You Submit

Work through this list. If every line is ticked, the project is ready.

**Table 16.1 — Pre-submission checklist.**

| Done? | Check |
|---|---|
| Yes | Python is installed and 'python --version' works in the command window. |
| Yes | The library is installed and prints a version starting with 4. |
| Yes | Photos were taken for every person you want recognised, about 60 each. |
| Yes | Training was run AFTER the photos were taken, and the output listed every person with a photo count above 0. |
| Yes | The recognition window opens and shows a green box with the correct name for someone in front of the camera. |
| Yes | Pressing A creates a row in the attendance file, and pressing A twice for the same person does not duplicate it. |
| Yes | An unknown person (someone who was never trained) shows a red UNKNOWN box. |
| Yes | You can explain the three stages and what LBPH stands for (Chapter 15). |

## 17. Optional: Let an AI Assistant Set It All Up

Everything up to Chapter 16 assumes you type each command yourself. There is a second option: let an AI coding assistant do the setup and the running for you.

One of these files came with the project:

```
face-recognition\AGENTS.md
```

This is an instruction file written for AI assistants. It is not a program and you never run it yourself. It tells the assistant what this project is, which mistakes to avoid, and exactly which steps to carry out in which order.

It works with Antigravity (Google's AI editor), and also with Cursor, GitHub Copilot, Claude Code and OpenAI Codex, because they all read the same file name. The file has no special formatting or settings needed — it is picked up automatically from the project folder.

**Table 17.1 — What can be automated, and what cannot.**

| What the assistant will do for you | What you still have to do |
|---|---|
| Check whether Python is installed and is the right version | Nothing |
| Install the library with the correct version pin | Nothing |
| Take the photos by driving the webcam, or copy your photos into the right folders | Sit in front of the camera, or choose a folder of photos |
| Train the model | Nothing |
| Verify the result and report which checks passed or failed | Nothing |
| Open the recognition window | Click the window, press A to mark attendance, Q to quit |

Being honest about the limit: the assistant cannot press the A and S keys for you, because those are live keyboard presses inside a camera window. It can build the photo dataset by itself, and it can start the recognition window, but for the final live run you must click the window and press the keys yourself.

**Table 17.2 — How to start it, depending on which tool you have.**

| If you use | Do this |
|---|---|
| Antigravity | Open the face-recognition folder as your project. AGENTS.md is read automatically. Then just type:  Follow AGENTS.md and set this project up. |
| Any other AI assistant | Open the face-recognition folder, then copy the whole prompt below into the chat and press send. |
| No AI assistant at all | Ignore this chapter and follow Chapters 3 to 9 in order. They work on their own. |

The complete prompt is reproduced below. It is generated directly from the AGENTS.md file, so the two are always identical.

```
# AGENTS.md — Face Recognition (OpenCV + LBPH)

You are setting up, running, and maintaining a small face-recognition project
on a **Windows** laptop. The user may be completely new to computers. Work
through the phases below **in order**, one at a time, and do not skip ahead.
```

## What this project is

```
A webcam program that recognises faces. Three scripts, run in order:

| Script | Stage | Produces |
|---|---|---|
| `capture_faces.py` | 1. Take photos | `dataset/<person>/*.jpg` |
| `train_model.py` | 2. Train the model | `trainer.yml`, `trainer.json` |
| `recognize_faces.py` | 3. Recognise live | `attendance/<date>.csv` |
```

`config.py` holds shared settings. `requirements.txt` holds dependencies.

## Hard rules — do not break these

1. **Never install `opencv-python`.** Only `opencv-contrib-python`. The face recogniser (`cv2.face`) exists only in the contrib build. Installing the wrong one makes training crash.
2. **Never install version 5 or newer.** Version 5 ships without the cascade XML files the program needs, so detection fails with "No face cascade found". `requirements.txt` already pins `>=4.5,<5` — keep that pin.
3. **Never delete `dataset/`, `trainer.yml`, or `trainer.json` without asking the user first.** Those contain their photographs and hours of work.
4. **Never claim a step succeeded without running the command and reading its real output.** If you did not run it, say so.
5. **Do not "fix" the source code** unless the user asks. This project is working; your job is to install, run, and verify it.
6. **If a command fails, read the actual error text** before trying anything. Do not guess at fixes.

## Phase 0 — Preflight (run all of these, then report)

Run and capture the real output of each:

```powershell
python --version
python -c "import cv2; print(cv2.__version__)"
python -c "import cv2; print(hasattr(cv2,'face'))"
Get-ChildItem -Name
```

Report to the user, in plain language:
- Is Python installed? (needs 3.9 or newer)
- Is `cv2` installed, and is it version 4.x?
- Does `cv2.face` exist? (must print `True`)
- Are all six project files present?

**If Python is missing entirely**, stop and give these instructions. Do not try to install it yourself — it needs a human clicking through a GUI installer:
1. Go to https://www.python.org/downloads/ and download the Windows installer.
2. **Tick "Add python.exe to PATH" on the first screen.** If this is missed, everything fails and must be redone.
3. Install, then re-run Phase 0.

**If `cv2` reports version 5 or newer**, fix it with:
`python -m pip install --force-reinstall "opencv-contrib-python>=4.5,<5"`

## Phase 1 — Install dependencies

```powershell
python -m pip install -r requirements.txt
```

This takes a few minutes and prints a lot of text. That is normal. Wait for it
to finish. Do not interrupt it.

Then re-run the Phase 0 cv2 checks and confirm the version starts with `4`.

## Phase 2 — Get training photos

**You must ask the user which method they want before doing anything here.**

### Option A — Drive the webcam automatically (user must be present)

Ask for each person's name, one at a time. For each name:

1. Confirm the Windows Camera app and any video call are closed.
2. Open the camera with `cv2.VideoCapture(0)` from the project folder.
3. Loop: read a frame, mirror it with `cv2.flip(frame, 1)`, convert to
   grayscale, `cv2.equalizeHist`, then detect a face with the cascade from
   `get_face_detector()` in `capture_faces.py` (import it — do not re-implement
   the cascade list).
4. When a face is found, crop the **largest** face, apply a small random
   translation and a small random brightness/contrast change, and save it as
   `dataset/<name>/img_NNN.jpg`. The jitter imitates head movement between
   shots and measurably improves accuracy.
5. Keep the last ~5 clean, unjittered crops aside as held-out test frames.
6. Stop after 60 saved photos. Tell the user to stay still and face the camera,
   and to move slightly between shots is NOT needed since you jitter yourself.

Say clearly what you are about to do before opening the camera, because the
user must be sitting in front of it.

### Option B — Use photos the user already has

Ask for the folder path. For each person, create `dataset/<name>/` and copy
their `.jpg` files in. If photos are `.png` or `.heic`, convert to `.jpg` first.
20+ clear, front-facing, well-lit photos per person is a reasonable minimum.

### Notes for both options
- Folder name becomes the on-screen label, so no spaces and consistent spelling.
- Remove obvious junk: photos with no face, heavy blur, or the face tiny in
  frame. `train_model.py` uses the whole image when it cannot find a face, so
  a photo of a wall actively harms the model.

## Phase 3 — Train

```powershell
python train_model.py
```

Read the output. It prints one line per person with a photo count. **Note these

counts** — you need them for Phase 4.

## Phase 4 — Verify (do not skip; report honestly)

Check every one of these and report PASS or FAIL for each:

1. `trainer.yml` exists and is not empty.
2. `trainer.json` exists and maps each person name to a number.
3. **Every person has a photo count greater than 0.** A `0` means that
   person's folder is empty, misplaced, or has the wrong file extension.
4. The `Classes:` line lists every person you intended to register.
5. Load the model and predict, without a webcam:
   ```python
   import cv2, json
   rec = cv2.face.LBPHFaceRecognizer_create()
   rec.read("trainer.yml")
   names = {int(k): v for k, v in json.load(open("trainer.json")).items()}
   img = cv2.imread(r"<one saved photo path>", cv2.IMREAD_GRAYSCALE)
   img = cv2.equalizeHist(cv2.resize(img, (300, 300)))
   print(rec.predict(img))
   ```
   The returned label must map back to the correct name, and the distance must
   be below `CONFIDENCE_LIMIT` (70) in `config.py`.
6. If you kept held-out test frames in Option A, predict each one and report
   the hit rate.

## Phase 5 — Run it live

```powershell
python recognize_faces.py
```

**This step needs the human and you cannot do it for them.** Tell the user:

- Click the camera window, then press **A** to mark attendance, **Q** to quit.
- A **green** box with a name means recognised. A **red UNKNOWN** box means the
  person is not in the training data.
- Afterwards check `attendance/<date>.csv` contains the rows.

## Phase 6 — Report

Finish with a short summary: what was installed, photos per person, the
held-out hit rate, whether every Phase 4 check passed, and anything the user
must do themselves. State plainly anything you did not verify.

## Troubleshooting — match the exact error text

| Error | Fix |
|---|---|
| `'python' is not recognized` | Python not installed, or PATH box was not ticked. Reinstall with the PATH box ticked. |
| `'pip' is not recognized` | Use `python -m pip`, not bare `pip`. |

| `No module named 'cv2'` | Install did not complete. Re-run Phase 1. |
| `module 'cv2' has no attribute 'face'` | Wrong package installed. `python -m pip uninstall opencv-python opencv-contrib-python`, then re-run Phase 1. |
| `No face cascade found` | OpenCV 5 is installed. Reinstall with `"opencv-contrib-python>=4.5,<5"`. |
| `Could not open the webcam` | Close the Windows Camera app. Check Settings → Privacy → Camera. If two cameras exist, set `CAMERA_INDEX = 1` in `config.py`. |
| `No dataset folder` | Phase 2 was skipped. |
| `No face photos found in the dataset folder` | Photos are not directly inside `dataset/<person>/` or are not `.jpg`. |
| No box around a face | Face too far away (must be over ~100 px), too dark, or not facing the camera. |
| Always shows red UNKNOWN | Training was not re-run after photos were added, or photos are poor. Retrain. |
| Everything is UNKNOWN for a registered person | Their folder is not inside `dataset/`, or the folder name differs from what they type at the prompt. |

## File map

```
face-recognition/
  capture_faces.py     stage 1
  train_model.py       stage 2
  recognize_faces.py   stage 3
  config.py            shared settings
  requirements.txt     dependencies (pinned <5)
  AGENTS.md            this file
  README.md            short reference
  dataset/<person>/*.jpg      created in phase 2
  trainer.yml                 created in phase 3
  trainer.json                created in phase 3
  attendance/<date>.csv       created in phase 5
```